

Skill – 17

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano job_control.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc job_control.c -o job_control
himasree@LAPTOP-41COT5AA:~/OSLab$ ./job_control
```

```
=====
JOB CONTROL DEMONSTRATOR
=====
```

- ```

1. Start Background Job
2. List Active Jobs
3. Bring Job to Foreground (fg)
4. Exit

```

```
Enter choice: 1
Enter execution time in seconds: 10
[1] 640 started in background.
```

- ```
-----
1. Start Background Job
2. List Active Jobs
3. Bring Job to Foreground (fg)
4. Exit
-----
```

```
Enter choice: Child 640 started.
```

- ```

1. Start Background Job
2. List Active Jobs
3. Bring Job to Foreground (fg)
4. Exit

```

```
Enter choice: 1
Enter execution time in seconds: 10
[1] 640 started in background.
```

- ```
-----
1. Start Background Job
2. List Active Jobs
3. Bring Job to Foreground (fg)
4. Exit
-----
```

```
Enter choice: Child 640 started.
Child 640 finished.
```

```
1
Enter execution time in seconds: 20
[2] 671 started in background.
Child 671 started.
```

- ```

1. Start Background Job
2. List Active Jobs
3. Bring Job to Foreground (fg)
4. Exit

```

```
Enter choice: |
```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ cat job_control.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 10

typedef enum
{
 RUNNING,
 STOPPED,
 DONE
} JobState;

typedef struct
{
 int job_id;
 pid_t pid;
 pid_t pgid;
 JobState state;
 char command[100];
} Job;

Job jobs[MAX_JOBS];

```

```

/* =====
GET STATE NAME
===== */

const char *state_name(JobState state)
{
 switch (state)
 {
 case RUNNING:
 return "Running";

 case STOPPED:
 return "Stopped";

 case DONE:
 return "Done";

 default:
 return "Unknown";
 }
}

/* =====
ADD JOB
===== */

void add_job(pid_t pid, pid_t pgid, const char *command)
{

```

```

/* =====
FIND JOB BY JOB ID
===== */

int find_job(int job_id)
{
 for (int i = 0; i < job_count; i++)
 {
 if (jobs[i].job_id == job_id)
 {
 return i;
 }
 }

 return -1;
}

/* =====
REMOVE JOB
===== */

void remove_job(int index)
{
 for (int i = index; i < job_count - 1; i++)
 {
 jobs[i] = jobs[i + 1];
 }
}

```

```

/* =====
UPDATE JOB STATES
===== */

void update_jobs()
{
 int status;

 for (int i = 0; i < job_count; i++)
 {
 pid_t result = waitpid(
 jobs[i].pid,
 &status,
 WNOHANG | WUNTRACED
);

 if (result == jobs[i].pid)
 {
 if (WIFSTOPPED(status))
 {
 jobs[i].state = STOPPED;
 }
 else if (WIFEXITED(status) ||
 WIFSIGNALED(status))
 {
 jobs[i].state = DONE;
 }
 }
 }
}

```

```

/* =====
LIST JOBS
===== */

void list_jobs()
{
 update_jobs();

 printf("\n=====\\n");
 printf(" ACTIVE JOBS\\n");
 printf("=====\\n");

 if (job_count == 0)
 {
 printf("No jobs available.\\n");
 return;
 }

 printf("%-8s %-10s %-10s %-12s %s\\n",
 "Job ID",
 "PID",
 "PGID",
 "State",
 "Command");

 printf("-----\\n");

 for (int i = 0; i < job_count; i++)
 {

```

```

/* =====
CREATE BACKGROUND JOB
===== */

void start_background_job(int seconds)
{
 pid_t pid = fork();

 if (pid < 0)
 {
 perror("fork");
 return;
 }

 if (pid == 0)
 {
 /*
 Child creates its own process group.
 */

 setpgid(0, 0);

 printf("Child %d started.\\n", getpid());

 sleep(seconds);

 printf("Child %d finished.\\n", getpid());

 exit(0);
 }

```

```

 case 3:

 printf("Enter Job ID to foreground: ");
 scanf("%d", &job_id);

 foreground_job(job_id);

 break;

 case 4:

 update_jobs();

 printf("\\nExiting job control program.\\n");

 return 0;

 default:

 printf("Invalid choice.\\n");

 }

 return 0;
}

```